

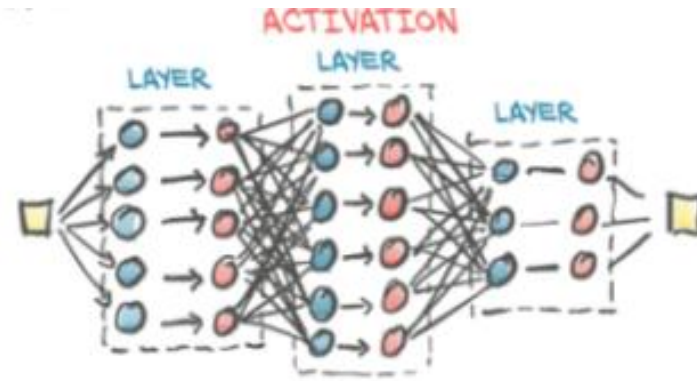
Deep Learning

LECTURE 4 –
THE MACHINE
LEARNING WORKFLOW

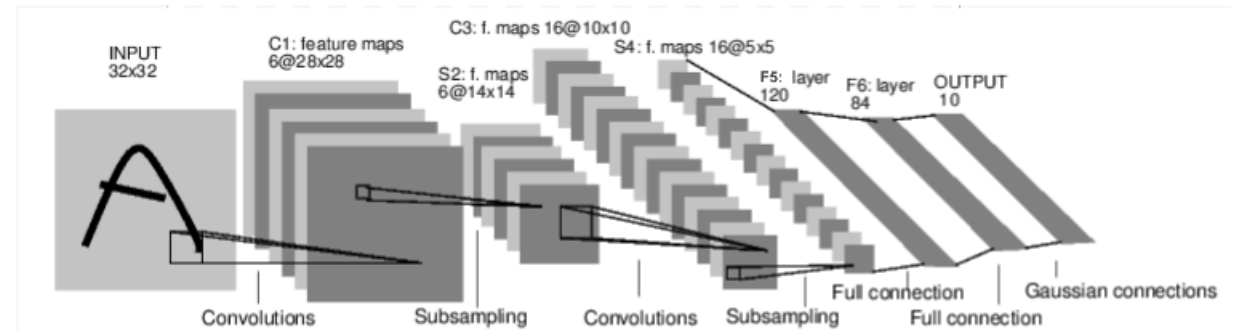
Kevin Bryson

We have examined two types of Deep Neural Networks:

1) **Fully connected feed-forward networks:**



2) **ConvNets or Convolutional Neural Networks (CNNs)**



Overview

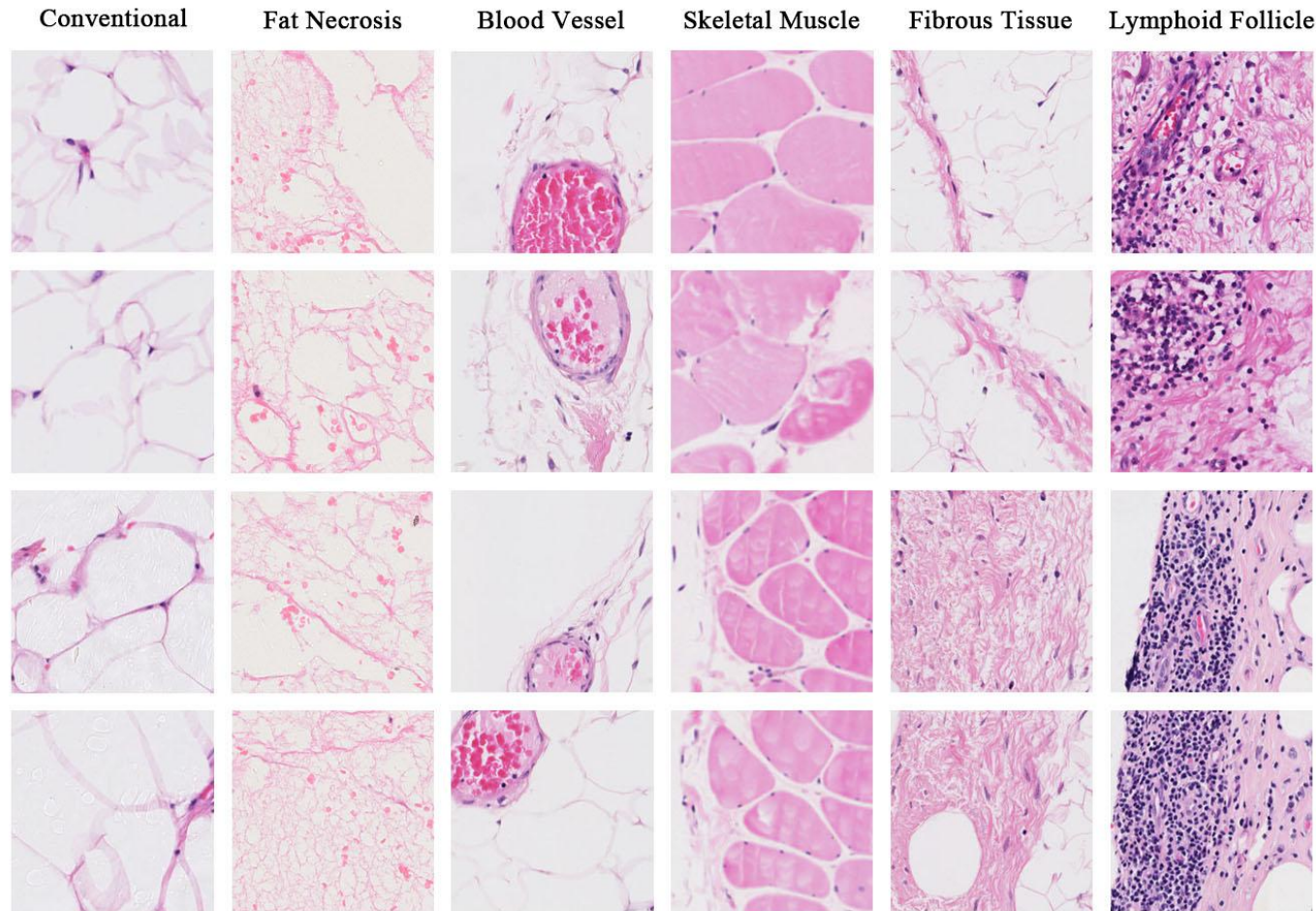
- I want to bring together all the key steps in a machine learning workflow that you would generally require for doing deep learning experiments.
(Such as the Jupyter Notebook Written Assessment for the course ... !)
- I will do this by presenting a motivational example in the area of digital pathology which is one of my research areas.
- A lot of this material will be very familiar from your ML course and I will go over it quickly – really just to quickly review it – but I will slow down when covering material which is more deep learning specific.

Important steps of a machine learning workflow

How should you plan your ML project?

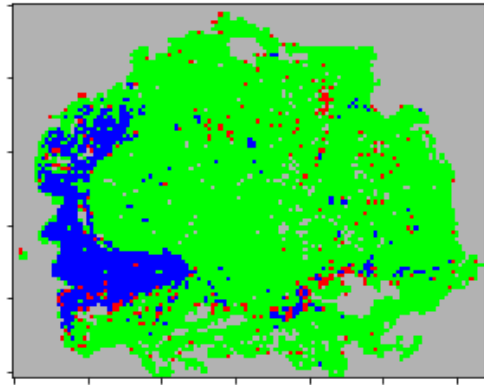
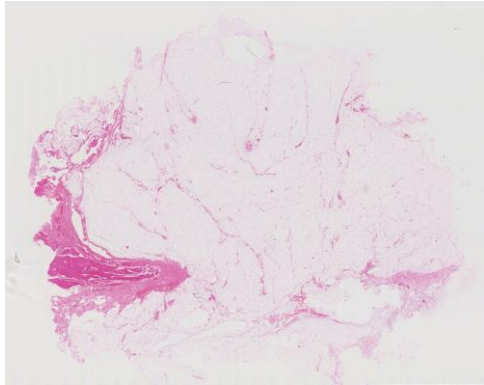
1. Define the problem and assemble a dataset
2. Choose a measure of success
3. Prepare the data
4. Decide on an evaluation protocol
5. Ensure you have ways to monitor and diagnose any issues during training
6. Quickly create **a simple model** to assess how it does against a baseline.
7. Scale up the model to get an “over-capacity” model !
8. Tune hyperparameters and regularise your model to optimize your solution.

Digital histopathology ... the digital analysis of microscope images for clinical purposes ...

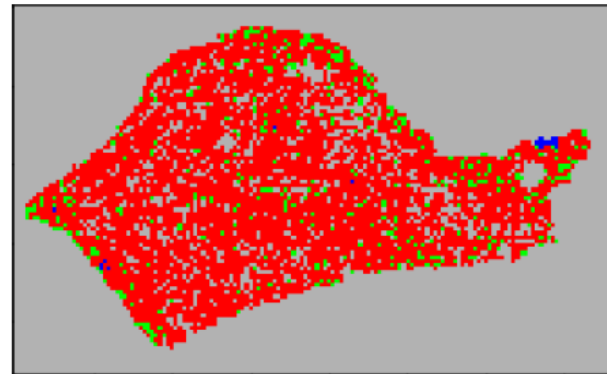
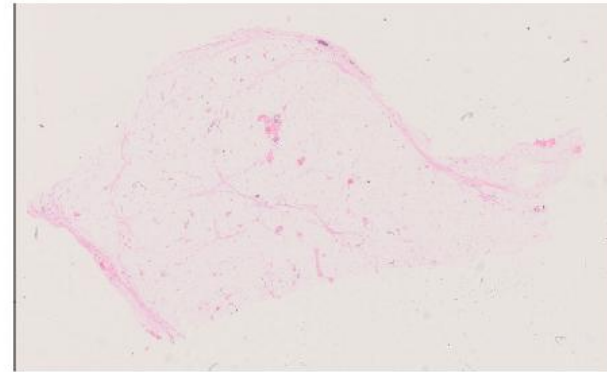


Motivational example: developing a CNN that can detect malignant liposarcoma (cancer)

Benign Lipoma



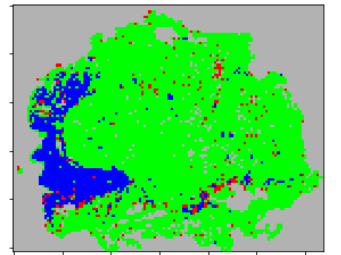
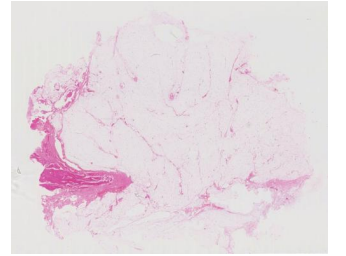
Malignant Liposarcoma



Benign lipoma Malignant Liposarcoma Muscle Background

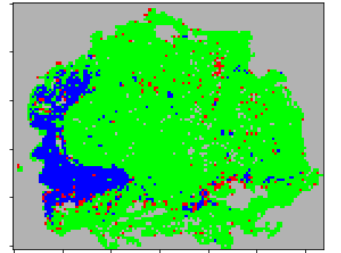
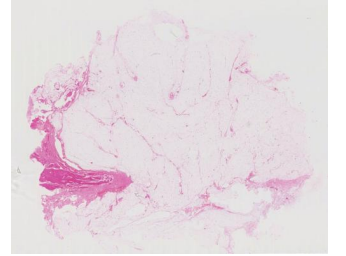
1. Defining the problem and assemble a dataset

- What are your inputs?
- What are you trying to predict? (Pathologist ? The CNN ?)
- What data is available? What can you go and gather?
- Define your problem more formally:
 - Binary or multiclass classification?
 - Scalar regression? Vector regression?
 - Clustering, generation of content, reinforcement learning?
- Think about your **implicit hypotheses**
 - That your outputs can be predicted from the inputs
 - That you have enough data to learn the relationship between inputs & outputs (Samples ? Training examples ?)



2. Choose a measure of success

- To control something you need to be able to measure it.
- What is your measure of success?
 - Accuracy? (Only really appropriate for balanced datasets.)
 - Precision & Recall ? (More appropriate for unbalanced datasets.)
 - Choice of measure affects approach (data augmentation needed?)
 - Should align with “higher level goals”:
 - Customer-retention rate
 - Profitability of a business
 - Efficiency ?
- Kaggle competition gives wide range of evaluation metrics
<https://kaggle.com/>



Model Performance Assessment

- For regression we often use
 - Mean-squared error
 - See more options
https://scikit-learn.org/stable/modules/model_evaluation.html#regression-metrics
- For classification, more options:
 - Confusion matrix
 - Accuracy
 - Cost-sensitive accuracy
 - Precision/recall
 - Area under ROC curve
- Lots of options in the sklearn.metrics tools
 - <https://scikit-learn.org/stable/api/sklearn.metrics.html#classification-metrics>

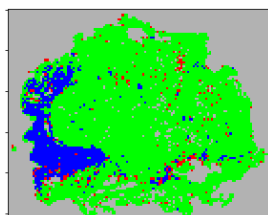
F1 score

- F1 score ranges between zero (a classifier with no real-world predictive power) and one (a classifier that has perfect predictions)

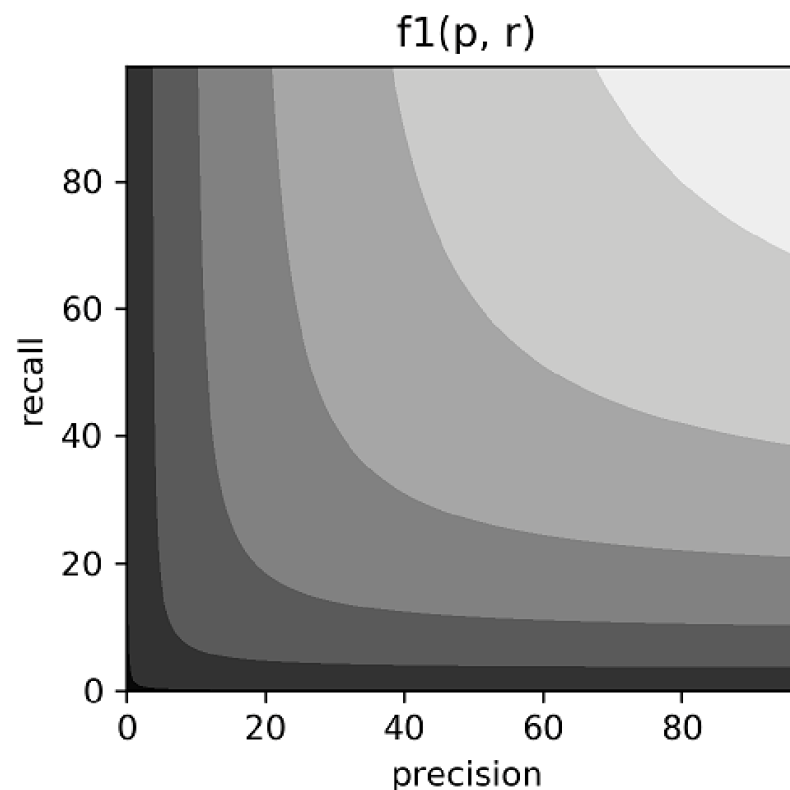
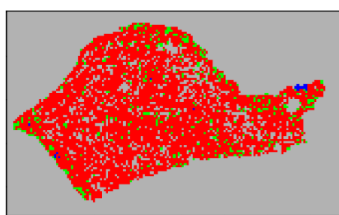
$$F1 = 2 \frac{\textit{precision} * \textit{recall}}{\textit{precision} + \textit{recall}}$$

- But need to be careful since high F1 score can vary a lot in terms of precision and recall.

Benign Lipoma

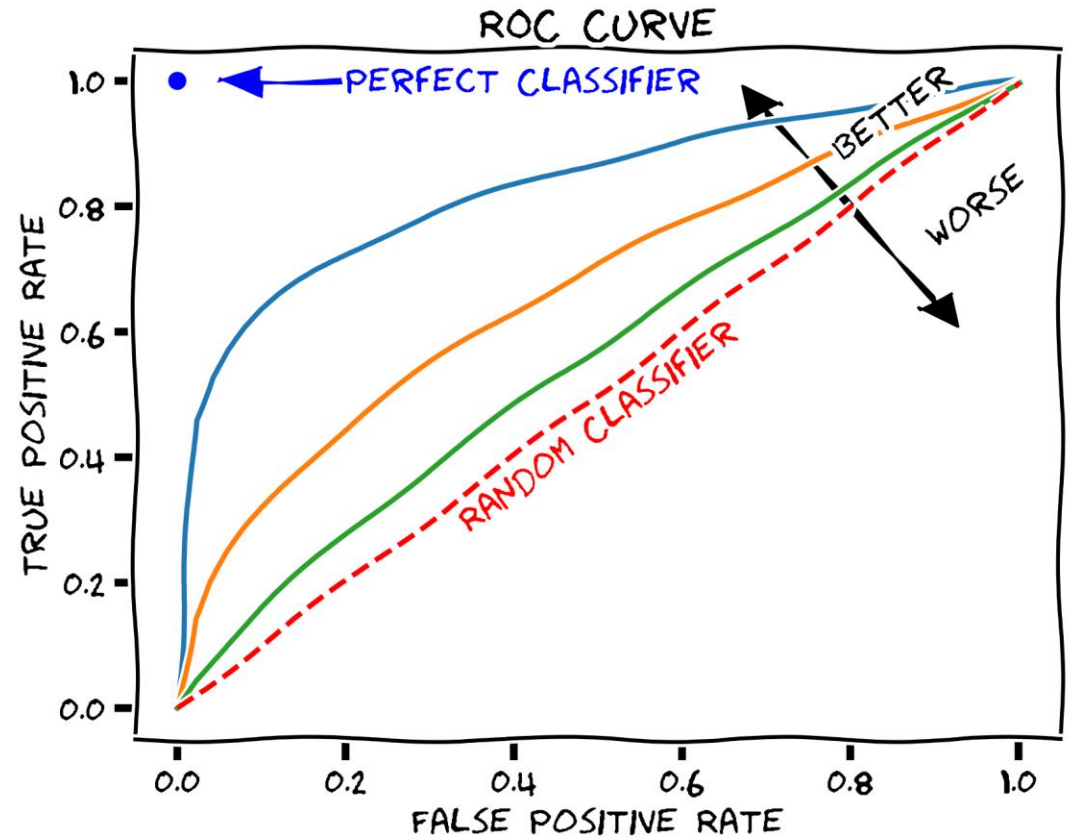


Malignant Liposarcoma



AUC – Area under ROC curve

- ROC – ‘Receiver Operating Characteristic’ (comes from RADAR engineering)
- Uses a combination of the true positive rate (recall) and false positive rate to create a curve.
- ROC curves require the classifier to return a probability of a +ve class so that different thresholds can be used to generate different points on the curve.



$$TPR = \frac{TP}{TP + FN}$$

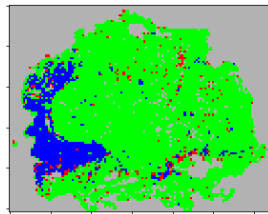
$$FPR = \frac{FP}{FP + TN}$$

Make no decision?

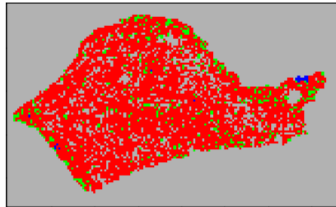
Sometimes a classifier should refuse to make a decision

- Useful when the classifier has confidence estimates
- When a false decision can be harmful
- Or when a human operator can take over

Benign Lipoma



Malignant Liposarcoma



3. Prepare your data (and choose an appropriate loss function)

You may have heard that deep learning models take “raw data” and they work out the features of the data that are important (representation learning)

But domain-specific preprocessing data for deep learning is still important:

- Many feature-engineering approaches are domain-specific (e.g. images, text with text embedding, audio with spectral analysis, etc.)
- Think about how easy it would be to learn from different data representations (feature engineering)
- Turn your data into vectors in an appropriate way (one-hot encoding, binning data, normalization, standardization, etc.)
- Handle missing values
- Data Augmentation

Feature Engineering (Chollet 2018)

Raw data: pixel grid		
Better features: clock hands' coordinates	$\{x1: 0.7,$ $y1: 0.7\}$ $\{x2: 0.5,$ $y2: 0.0\}$	$\{x1: 0.0,$ $y2: 1.0\}$ $\{x2: -0.38,$ $2: 0.32\}$
Even better features: angles of clock hands	$\theta_1: 45$ $\theta_2: 0$	$\theta_1: 90$ $\theta_2: 140$

Figure 4.3 Feature engineering for reading the time on a clock

One-hot Encoding

- Used to convert nominal categorical information (e.g. 'eye colour') into binary vectors.
- In one-hot encoding, each token is represented by a vector of length N , where N is the size of the vocabulary.
 - E.g. for a categorical feature 'eye colour' which can be Blue, Brown, Green or Other we could use:
 - Blue = [1, 0, 0, 0]
 - Brown = [0, 1, 0, 0]
 - Green = [0, 0, 1, 0]
 - Other = [0, 0, 0, 1]
- This increases the dimensionality of the feature vector, but is better than a numerical encoding (e.g. Blue =1, Brown=2, Green=3, Other=4) as this suggests an ordering of the categories which is not there. This type of encoding would be more suitable for ordinal categorical features (e.g. age range).

Binning/Bucketing

- The opposite approach is converting a numerical value into ordinal (categorical) ones
- Binning is process of converting continuous features into multiple binary features ('bins' or 'buckets') based on a value range.
- E.g. convert ages from real-valued features to discrete bins, 0-5 years, 6-10 years, etc.
- This can be a way of introducing prior knowledge into the ML task.

Normalisation

- Normalisation is the process of converting a numerical range of values into a 'standard' range, typically in the range $[-1, 1]$ or $[0, 1]$, such as:

$$\hat{x}_j = \frac{x_j - \min(x_j)}{\max(x_j) - \min(x_j)}$$

- This can speed up and stabilise learning (e.g. imagine a network with some feature inputs in 10^{-6} range while others are in 10^{+6} . Their derivatives would be in very different ranges).
- Having the same general range for inputs also helps us initialise our range of hyperparameters like the learning rate.

Standardisation

- You might choose to rescale feature values so that they have the properties of a standard normal distribution with mean 0 and standard deviation 1.

$$\hat{x}_j = \frac{x_j - \mu_j}{\sigma_j}$$

- No hard and fast rules for standardisation vs normalisation, but often used when data is close to Normal distribution, or when sets have some extreme outliers, as in that case normalisation can force the data into a very small range

Missing Data

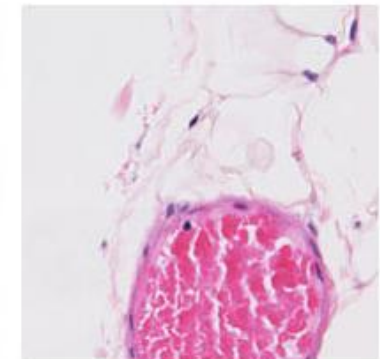
Often real data has missing values. Common approaches to handle:

- Throwing away examples with missing features
- Using a learning algorithm which can cope with missing data
- Using a *data imputation* technique
 - E.g. replace with mean value of that feature (might be dangerous)
 - Use other features to predict the missing data (data imputation)
 - Replace with an arbitrary value outside the normal range, in the hope that the learning algorithm learns to treat this differently.

Data Augmentation

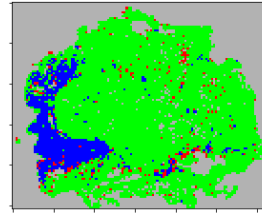
- Data augmentation is the process of generating more data by transforming the current data that you have **in acceptable ways**.
- For instance, with images, applying rotations, resizing or translating them, or changing contrast levels or image colouring.
- This helps improve generalization since it is a regularization technique (more about this in the next lecture) – but also makes the network robust to these transformations.
- Common approaches
 - Mirroring the image up-down, left-right, and/or front-back
 - Shifting the image around by a few pixels
 - Rotating the image around the head-foot axis
 - Adding noise to the image
 - Scaling the image up or down ... **but not for histopathology!**
(since 40x microscope image needs to stay at 40x)
So need to be careful that you create realistic augmentation for *your problem* ...

Blood Vessel

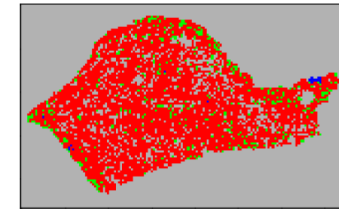


Class Imbalance

Benign Lipoma



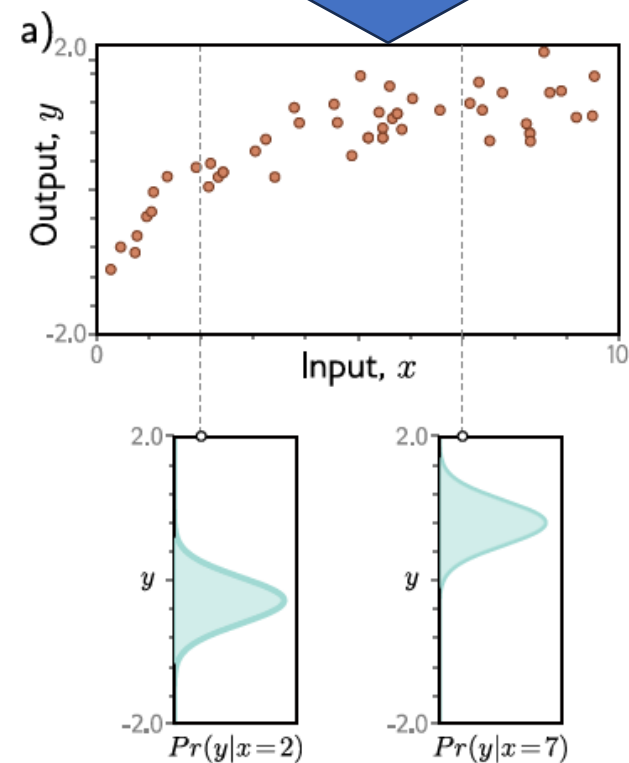
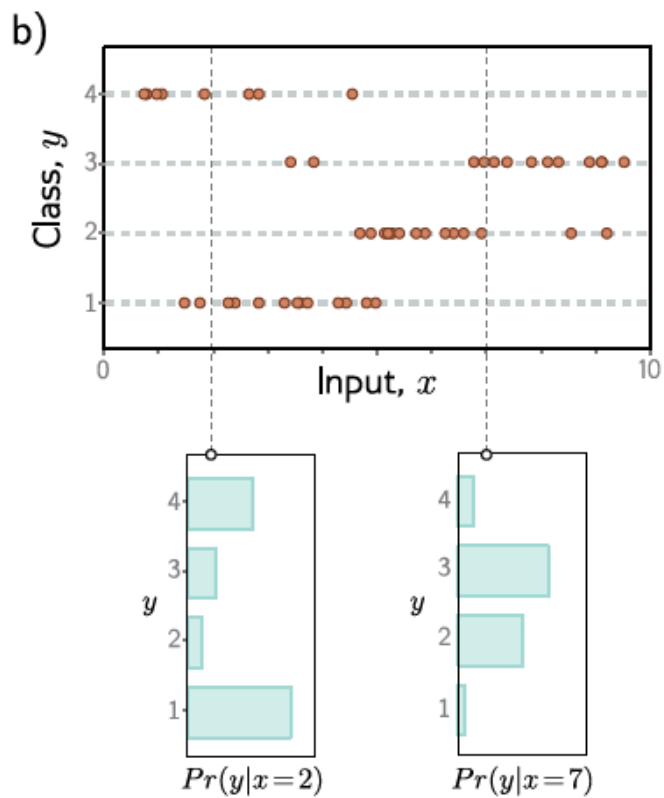
Malignant Liposarcoma



- In many real-world problems one class is much less frequent than another
- But often the minority classes are important (e.g. in clinical cases, there are more normal than abnormal samples, but the abnormal ones can be more dangerous).
- Can use data augmentation to balance the data by generating more minority class data.
- Or could simply use a weighted sampler to sample the classes more uniformly during training (<https://pytorch.org/docs/stable/data.html#torch.utils.data.Sampler>)
- Or could use a weighted loss function to give more weight to minority classes.

Target output type and associated loss (or cost) functions

Output Type	Output Distribution	Output Layer	Cost Function
Binary	Bernoulli	Sigmoid	Binary cross-entropy
Discrete	Multinoulli	Softmax	Discrete cross-entropy
Continuous	Gaussian	Linear	Gaussian cross-entropy (MSE)



How to construct loss functions

- Model predicts output y given input x

How to construct loss functions

- ~~• Model predicts output y given input x~~

How to construct loss functions

- ~~Model predicts output y given input x~~

- Model predicts a conditional probability distribution:

$$Pr(\mathbf{y}|\mathbf{x})$$

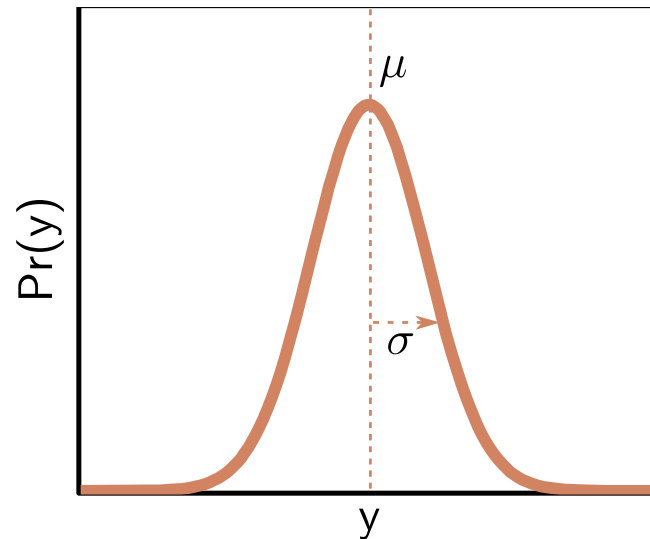
over outputs $\mathbf{y}$ given inputs $\mathbf{x}$.

- Loss function aims to make the outputs have high probability

How can a model predict a probability distribution?

1. Pick a known distribution (e.g., normal distribution) to model output y with parameters θ

e.g., the normal distribution $\theta = \{\mu, \sigma^2\}$



2. Use model to predict parameters θ of probability distribution

Maximum likelihood criterion

$$\begin{aligned}\hat{\phi} &= \operatorname{argmax}_{\phi} \left[\prod_{i=1}^I \operatorname{Pr}(\mathbf{y}_i | \mathbf{x}_i) \right] \\ &= \operatorname{argmax}_{\phi} \left[\prod_{i=1}^I \operatorname{Pr}(\mathbf{y}_i | \boldsymbol{\theta}_i) \right] \\ &= \operatorname{argmax}_{\phi} \left[\prod_{i=1}^I \operatorname{Pr}(\mathbf{y}_i | \mathbf{f}[\mathbf{x}_i, \phi]) \right]\end{aligned}$$

When we consider this probability as a function of the parameters ϕ , we call it a **likelihood**.

Maximum log likelihood

$$\begin{aligned}\hat{\phi} &= \operatorname{argmax}_{\phi} \left[\prod_{i=1}^I Pr(\mathbf{y}_i | \mathbf{f}[\mathbf{x}_i, \phi]) \right] \\ &= \operatorname{argmax}_{\phi} \left[\log \left[\prod_{i=1}^I Pr(\mathbf{y}_i | \mathbf{f}[\mathbf{x}_i, \phi]) \right] \right] \\ &= \operatorname{argmax}_{\phi} \left[\sum_{i=1}^I \log \left[Pr(\mathbf{y}_i | \mathbf{f}[\mathbf{x}_i, \phi]) \right] \right]\end{aligned}$$

Now it's a sum of terms, so doesn't matter so much if the terms are small

Minimizing negative log likelihood

- By convention, we minimize things (i.e., a loss)

$$\begin{aligned}\hat{\phi} &= \operatorname{argmax}_{\phi} \left[\sum_{i=1}^I \log \left[\operatorname{Pr}(\mathbf{y}_i | \mathbf{f}[\mathbf{x}_i, \phi]) \right] \right] \\ &= \operatorname{argmin}_{\phi} \left[- \sum_{i=1}^I \log \left[\operatorname{Pr}(\mathbf{y}_i | \mathbf{f}[\mathbf{x}_i, \phi]) \right] \right] \\ &= \operatorname{argmin}_{\phi} \left[\mathbf{L}[\phi] \right]\end{aligned}$$

Target output type and associated loss (or cost) functions

Output Type	Output Distribution	Output Layer	Cost Function
Binary	Bernoulli	Sigmoid	Binary cross-entropy
Discrete	Multinoulli	Softmax	Discrete cross-entropy ★
Continuous	Gaussian	Linear	Gaussian cross-entropy (MSE)



Note special considerations are needed in PyTorch using the CrossEntropyLoss which assumes a **linear layer** as output (since internally the loss has a LogSoftMax).

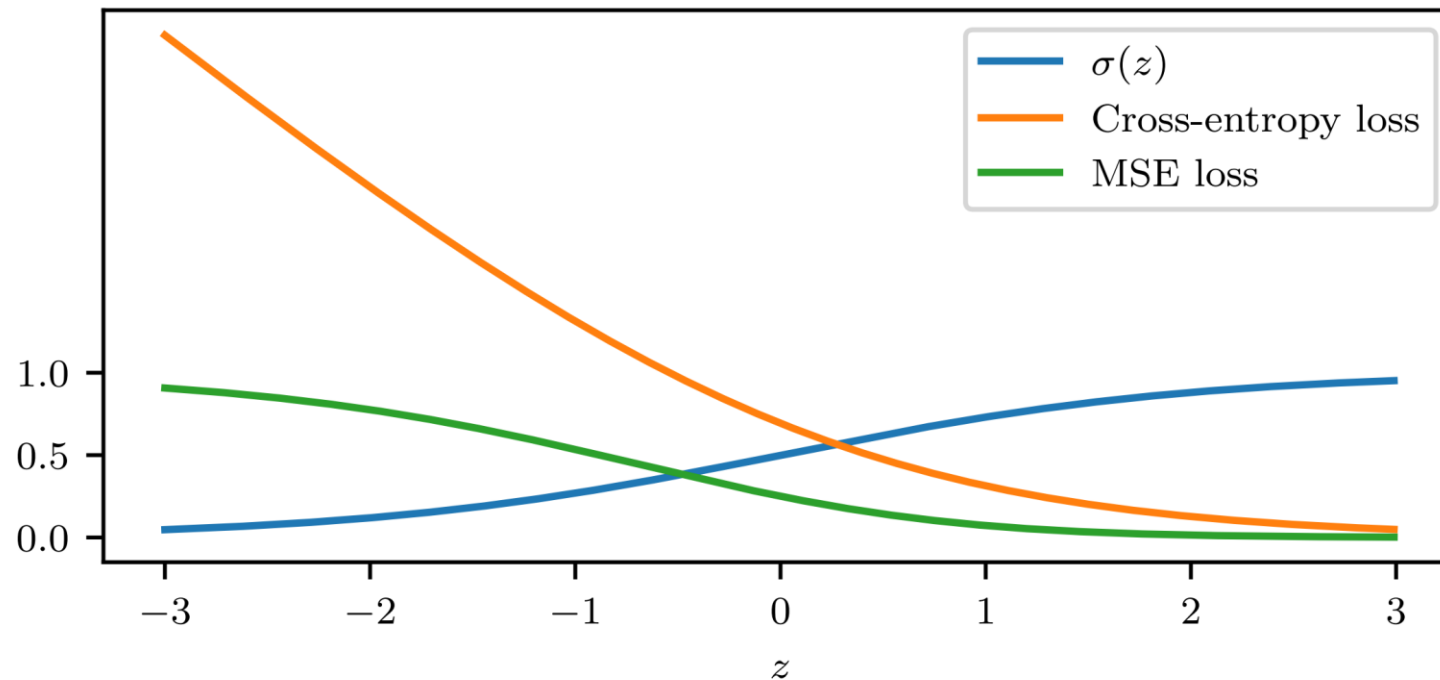
Also Negative Log Likelihood loss (NLLloss) in PyTorch assumes a **LogSoftMax** in the output layer of the model !

But a SoftMax is needed on the output layer to produce proper probability values if you wish these for predictions !

More detail about loss functions given in Chapter 5 of Understanding Deep Learning and also Goodfellow Section 5.5.

Choice of output layer and loss function matters! Inefficient optimization if wrong one is chosen!

Sigmoid output with target of 1



4. Decide on an evaluation protocol

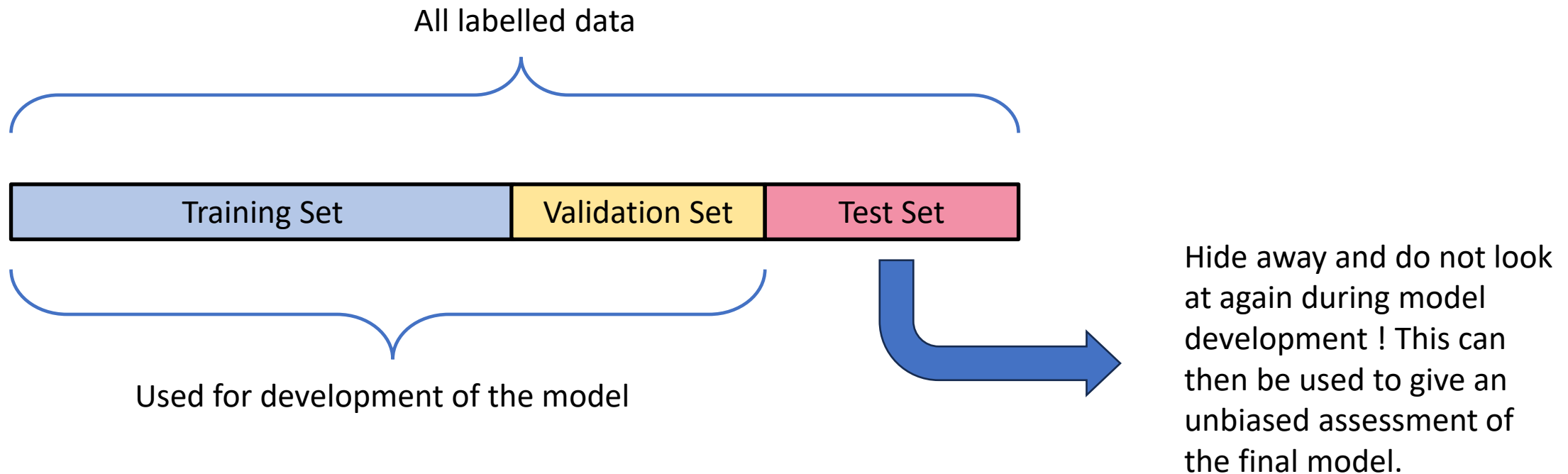
How do you measure current progress in developing your deep learning model ?

How do you do hyperparameter tuning or optimization ?

How do you know what the accuracy of your final model is ?

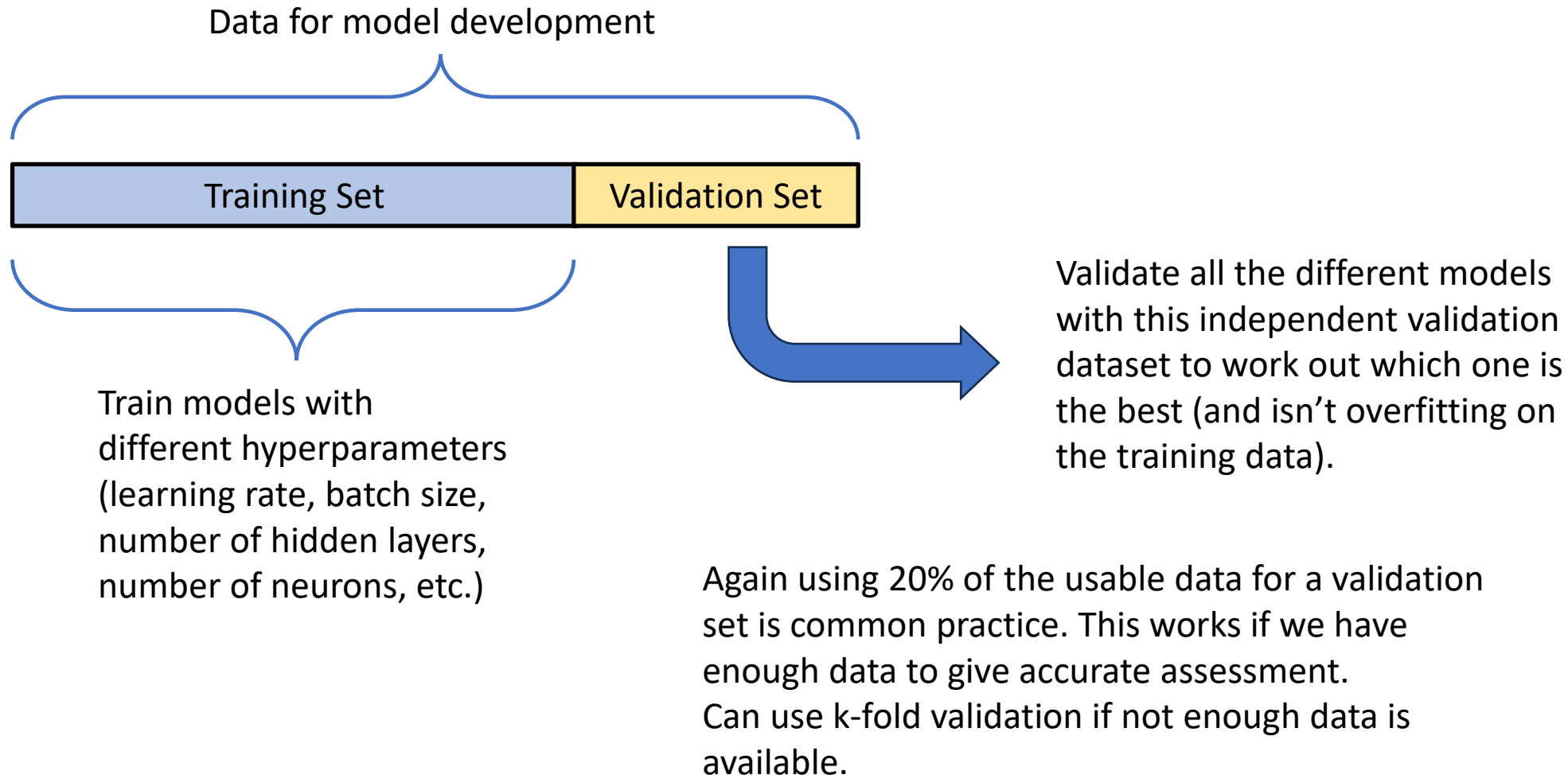
- Often split labelled data into three parts:
 - **Training set** used to train the model.
 - **Validation set** to test different models and optimize hyperparameters.
 - **“Out of sample” test set** to determine the unbiased accuracy of the one optimal final model.
- Once you have a fully optimized model, you get its final unbiased determination of accuracy **by testing once** on the test set.
- It is then common to train your model from scratch **using all the data** so you get the most accurate model to deploy (which should be at least as accurate as your test accuracy indicates – or even more accurate).

“Out of sample” independent test set ...



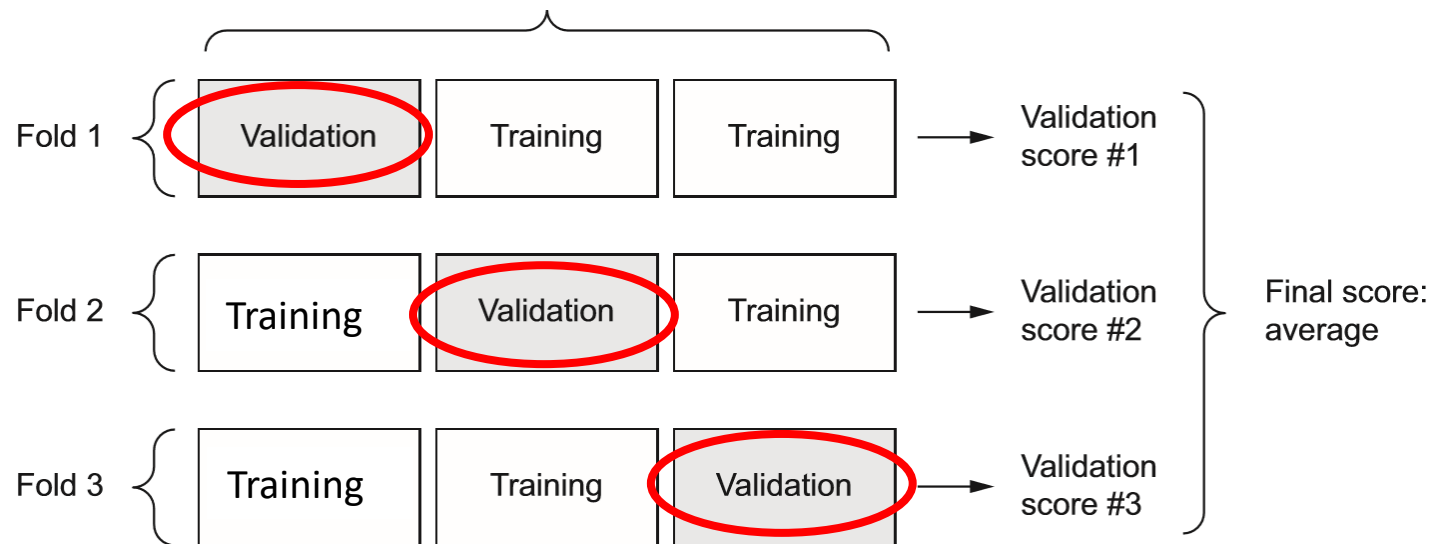
Common approach is to randomly sample 20% of the data to act as an unseen “out of sample” test dataset (but depends on how accurate we want the result).

Using a single validation dataset



K-fold validation (example showing 3-fold)

Data for model development split into 3 partitions



This is more computationally demanding than using a single validation set (since you have to train the model multiple times).

But it makes more use of the data to determine accurate validation scores when you have limited amounts of data available.

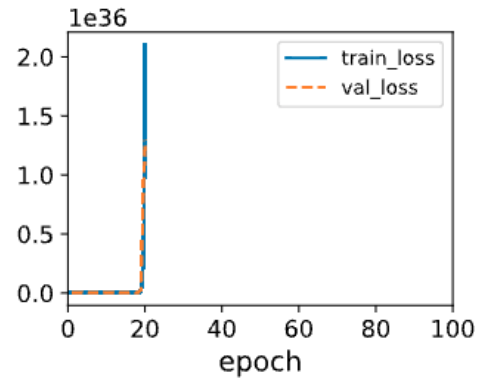
5. Ensure you have ways to monitor and diagnose any issues during training

There are lots of issues that can arise with machine learning that you need to be able to diagnose and solve. This requires a good understanding of the theory and how learning is happening:

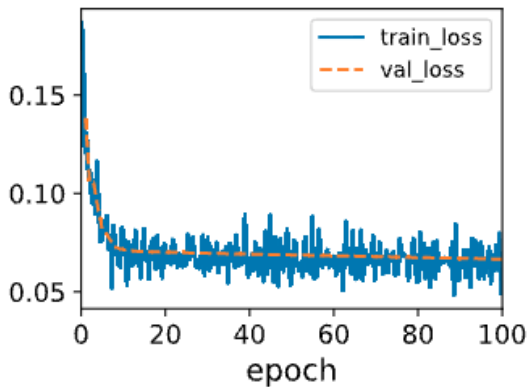
- **Underfitting ?** (Model is not complex enough to properly represent the ground truth relationship. Model has insufficient 'capacity'.)
- **Overfitting ?** (Model capacity is too high for the amount of training data used – overly fitted to the training data – cannot generalize to the test data.)
- **Convergence issues ?** (Enough epochs? Vanishing gradient ?)
- **Bottlenecks in computational performance ?** (Need to be solved before spending hours of GPU time on hyperparameter optimization.)

Monitoring your loss curves (or accuracy curves) learning rate is a key hyperparameter to tune

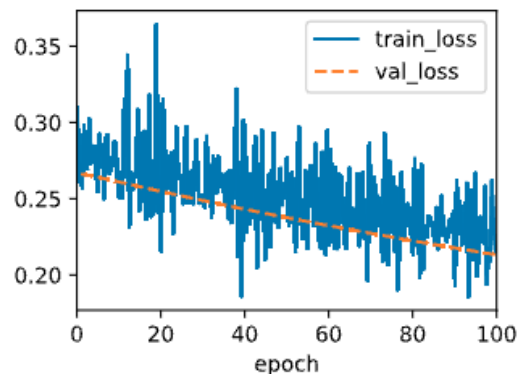
Example curves
showing too
high, low and
just right
learning rates



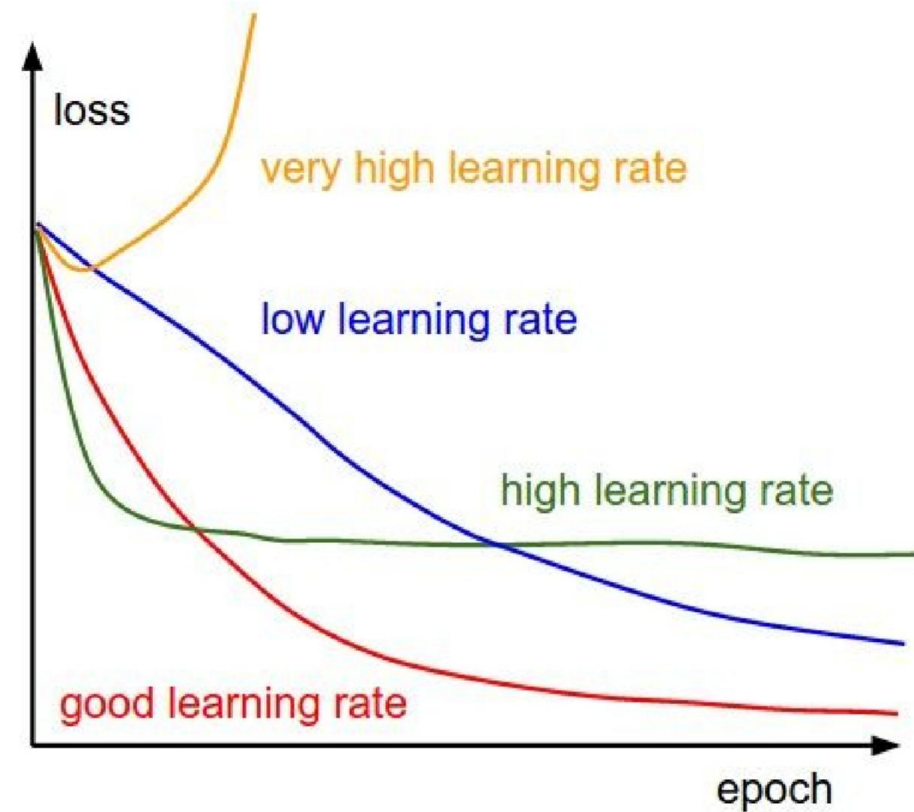
Ir = 2.0



Ir = 0.01

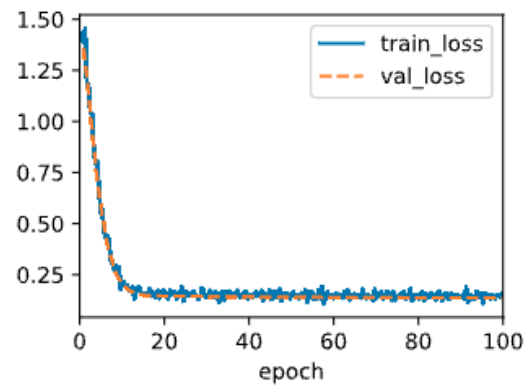


Ir = 0.00001

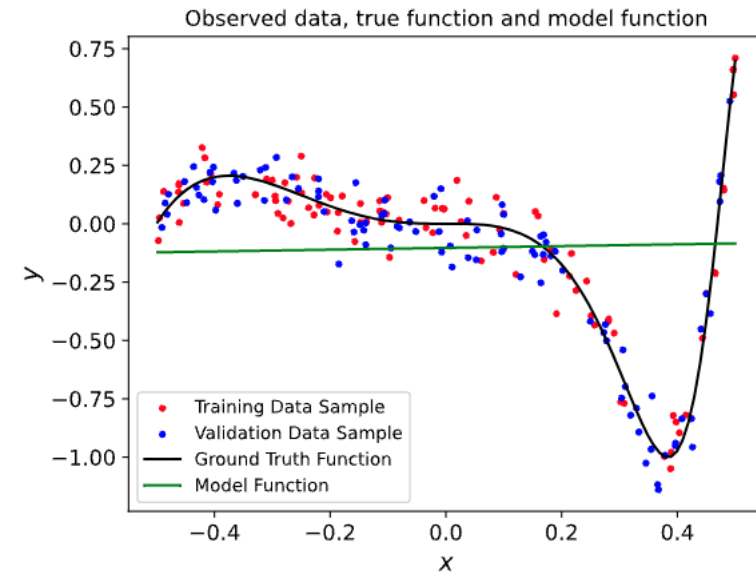


Monitoring your loss curves – underfitting / overfitting

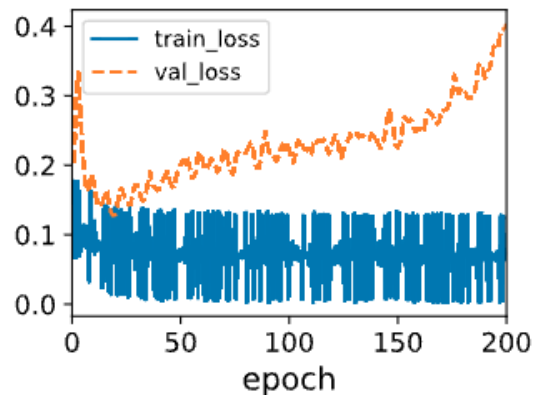
1 hidden layer with 1 neuron!



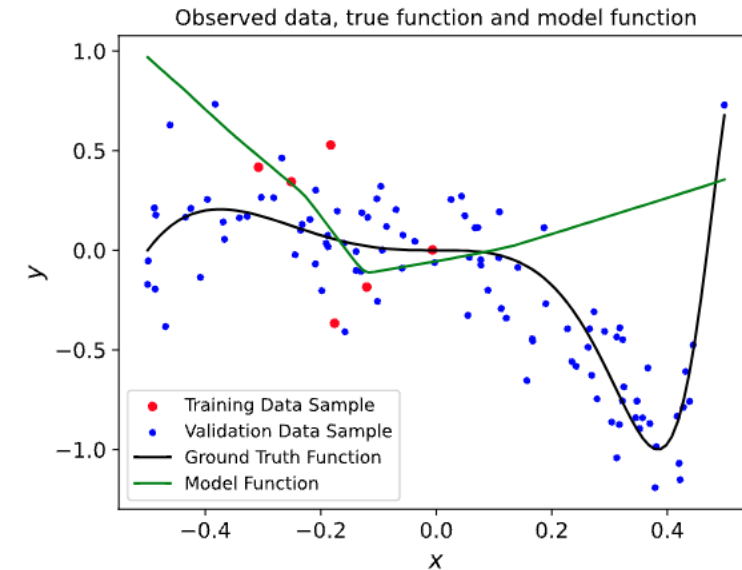
UNDERFITTING



2 hidden layers with 50 neurons
each (and only 6 training points!)



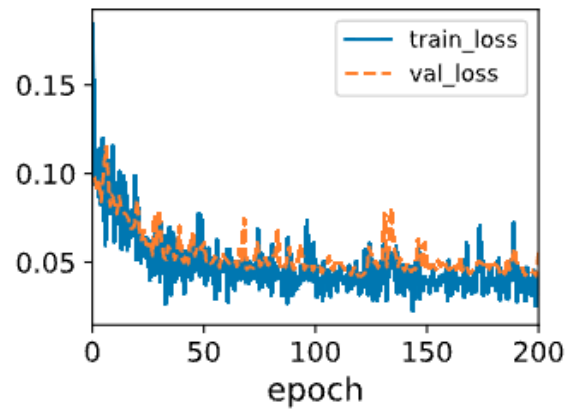
OVERFITTING



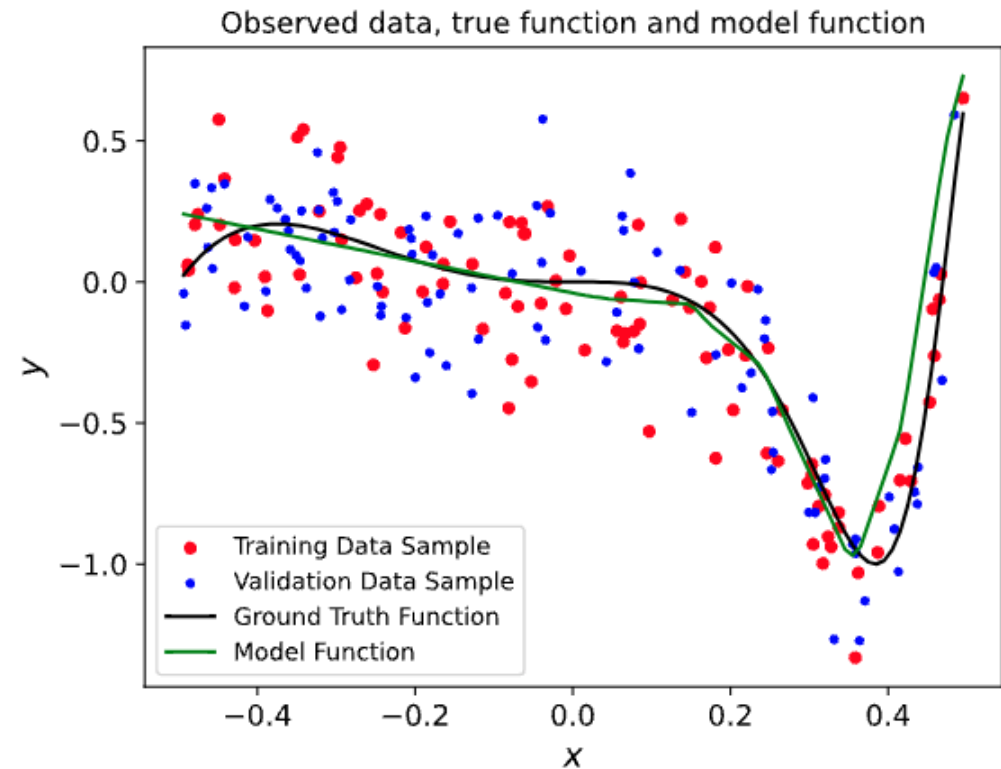
And the porridge is just right (Goldilocks Principle)

2 hidden layers with 50 neurons each
(with 100 training points!)

So exactly the same network as
before but just more training points.



JUST RIGHT



6. Quickly create a simple model to assess how it does against a baseline.

- Quickly prototypes to establish any issues with either the data or task and also provides a lower baseline to beat !
- Quickly tests the **implicit hypothesis** that the input features can actually be used for the prediction! (This is not always the case ...)
- For instance, in MNIST (10 digit classes), can the model beat the random baseline accuracy of 10% ? (Assuming balanced test set ...)
- This simple model may be sufficiently accurate that further work is not needed (i.e. data is linearly separable and so there is nothing to gain from using massive non-linear models).

7. Scale up the model to get an “over-capacity” model ! (Often more parameters than data points ...)

- But will this not just ‘overfit’ ? Not necessarily if appropriate normalization is used and also, we have the double-descent phenomenon (more about this in the next lecture).
- An overfitted model establishes an upper bound on accuracy based on the training data ...
- But will this not just be an upper bound of 100% accuracy?
- Not always ... Bayes error rate due to noise in the data ... and also input features may not be informative for prediction ...
- Often get better test performance from high complexity deep learning models that are regularized appropriately than lower complexity models (more in next lecture).

8. Tune hyperparameters and regularise your model to optimize your solution

- Hyperparameter optimization is extremely important for deep learning models to obtain the best models.
- Regularization is also very important for deep learning models and this will be discussed extensively next week (including L1 and L2 regularisation, Dropout, etc.). Regularization parameters often need hyperparameter tuning.
- **Beware!** Every time you use feedback from your validation set to change your model (i.e. hyperparameter optimization), you leak information from it into the model.

(This is why you need a final “out-of-sample test set” to determine the real accuracy of your final model ... and you test it only once !)

Hyperparameter optimisation

(This will be topic of Lab 5 ...)

- Lots of discrete decisions on architecture
 - How many layers? How many units? Which activation functions? Use Batch Normalisation? Use Dropout?
- Lots of continuous values to optimize
 - E.g. learning rate, momentum parameter, weight decay, how much dropout ?
- These can all be viewed as hyperparameters which should be optimised (however, unlike the model weights and biases, hyperparameters cannot be optimized by the learning procedure).
- Approaches
 - Manual optimization based on your understanding of how hyperparameter affect learning ...
 - Grid search (slow...)
 - Random search
 - Bayesian optimisation

Lot of hyperparameters to optimize based on understanding them ...

- The learning rate is vital to ensure whether we get effective learning.
- But key to the capacity of a model is the number of hidden units (or convolutional kernels) – **network architecture**.
- Regularization (weight decay, dropout, etc.) often used to moderate the model capacity.

Hyperparameter	Increases capacity when...	Reason	Caveats
Number of hidden units	increased	Increasing the number of hidden units increases the representational capacity of the model.	Increasing the number of hidden units increases both the time and memory cost of essentially every operation on the model.
Learning rate	tuned optimally	An improper learning rate, whether too high or too low, results in a model with low effective capacity due to optimization failure.	
Convolution kernel width	increased	Increasing the kernel width increases the number of parameters in the model.	A wider kernel results in a narrower output dimension, reducing model capacity unless you use implicit zero padding to reduce this effect. Wider kernels require more memory for parameter storage and increase runtime, but a narrower output reduces memory cost.
Implicit zero padding	increased	Adding implicit zeros before convolution keeps the representation size large.	Increases time and memory cost of most operations.
Weight decay coefficient	decreased	Decreasing the weight decay coefficient frees the model parameters to become larger.	
Dropout rate	decreased	Dropping units less often gives the units more opportunities to “conspire” with each other to fit the training set.	

Table 11.1: The effect of various hyperparameters on model capacity.

Learning rate is one of the most important hyperparameters which needs optimization

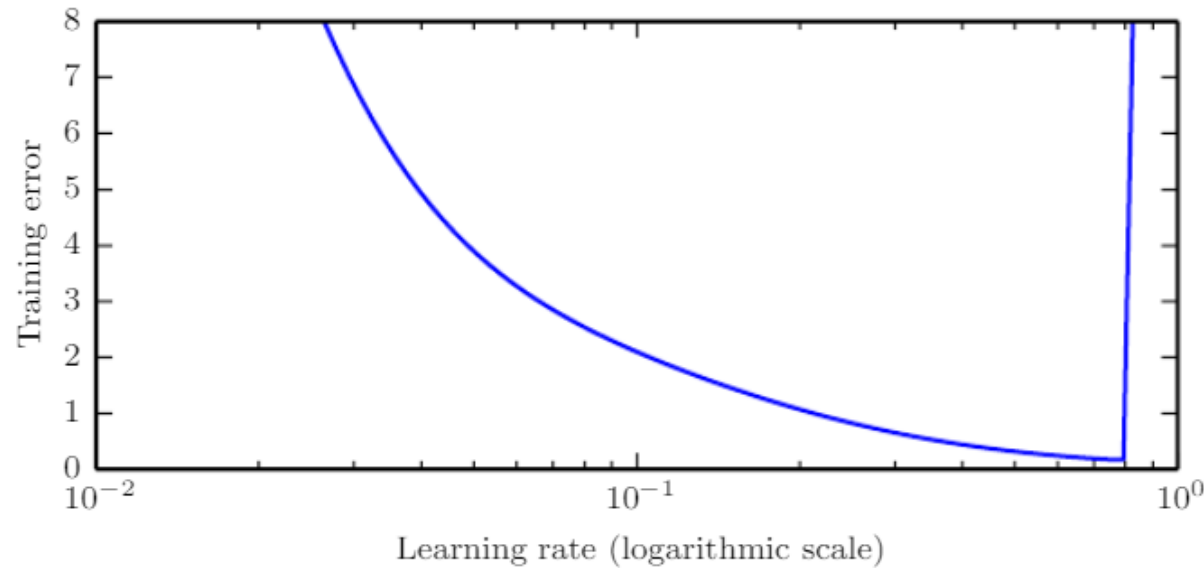
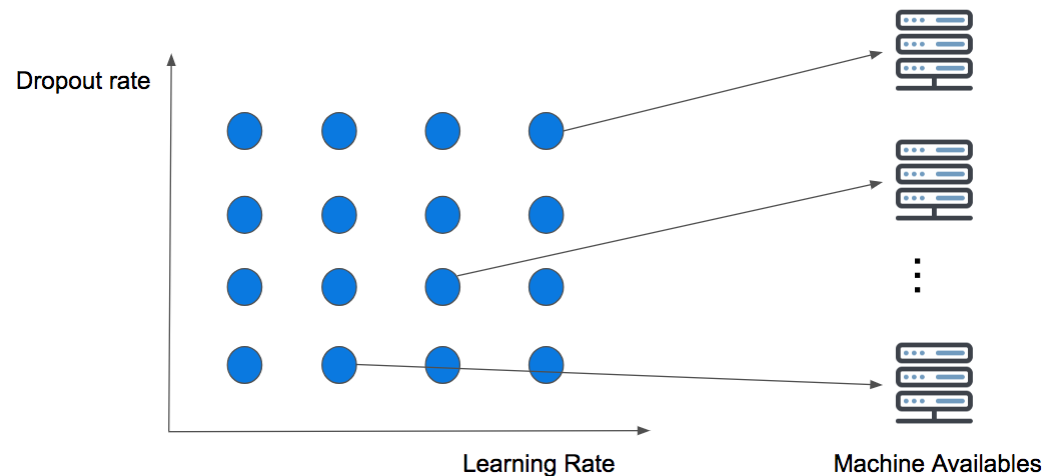


Figure 11.1: Typical relationship between the learning rate and the training error. Notice the sharp rise in error when the learning is above an optimal value. This is for a fixed training time, as a smaller learning rate may sometimes only slow down training by a factor proportional to the learning rate reduction. Generalization error can follow this curve or be complicated by regularization effects arising out of having too large or too small learning rates, since poor optimization can, to some degree, reduce or prevent overfitting, and even points with equivalent training error can have different generalization error.

Grid search

- Define a grid on n dimensions, where each of these maps for an hyperparameter. e.g. $n = (\text{learning_rate}, \text{dropout_rate}, \text{batch_size})$
- For each dimension, define the range of possible values: e.g. $\text{batch_size} = [4, 8, 16, 32, 64, 128, 256]$
- Calculate performance for all the possible configurations



Random search

- Often random search is more effective than grid search ...

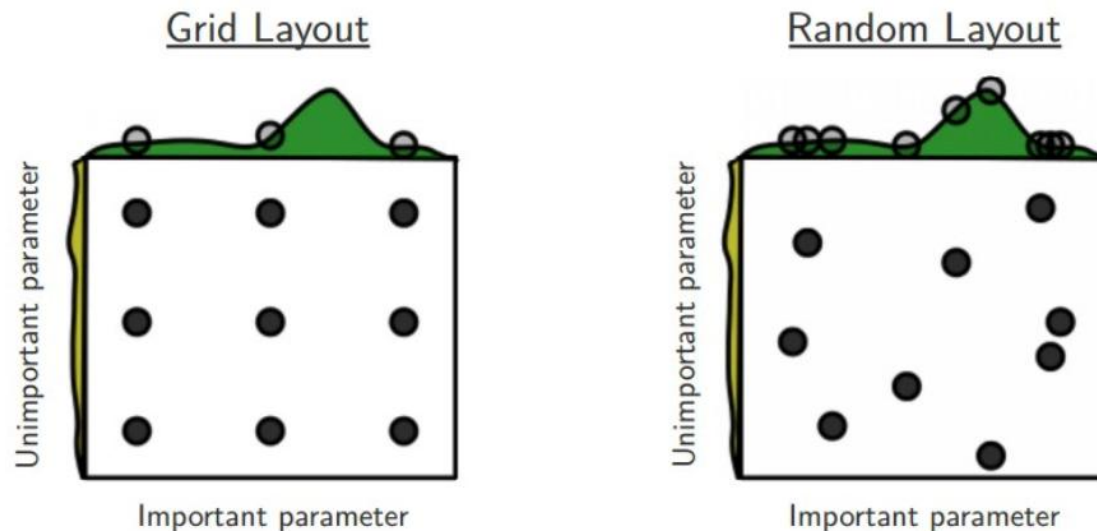
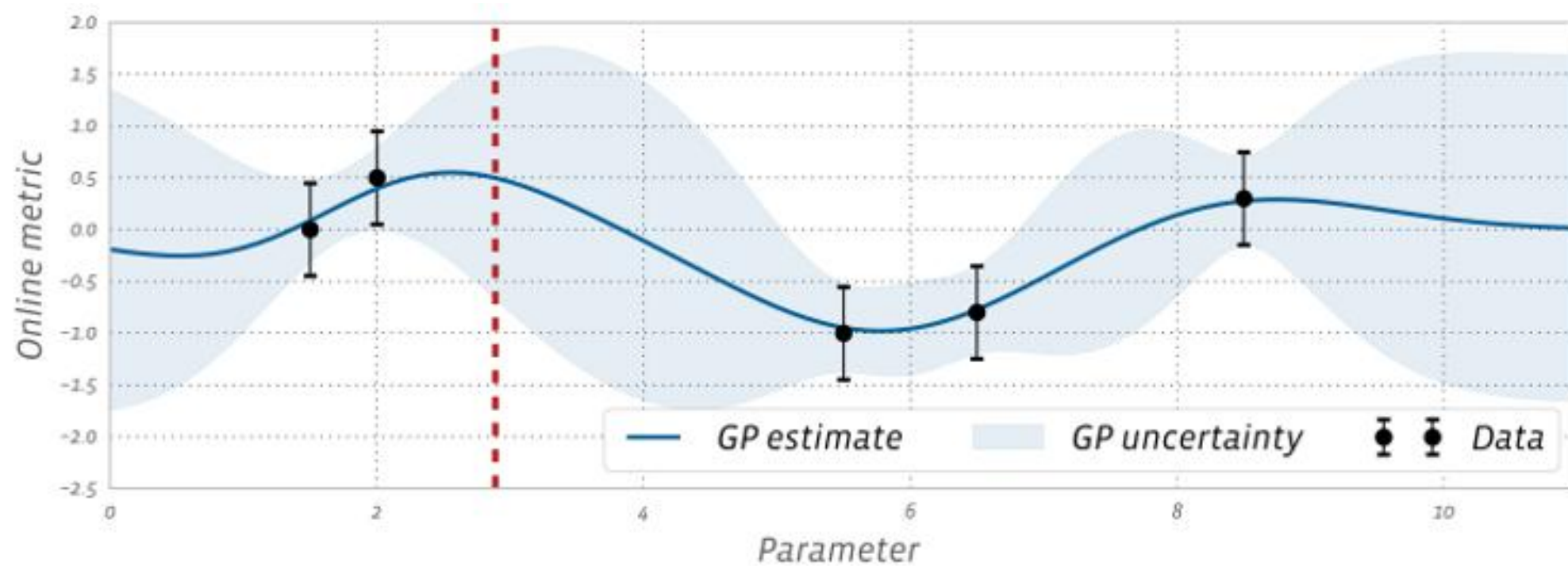


Image from <http://jmlr.csail.mit.edu/papers/volume13/bergstra12a/bergstra12a.pdf>

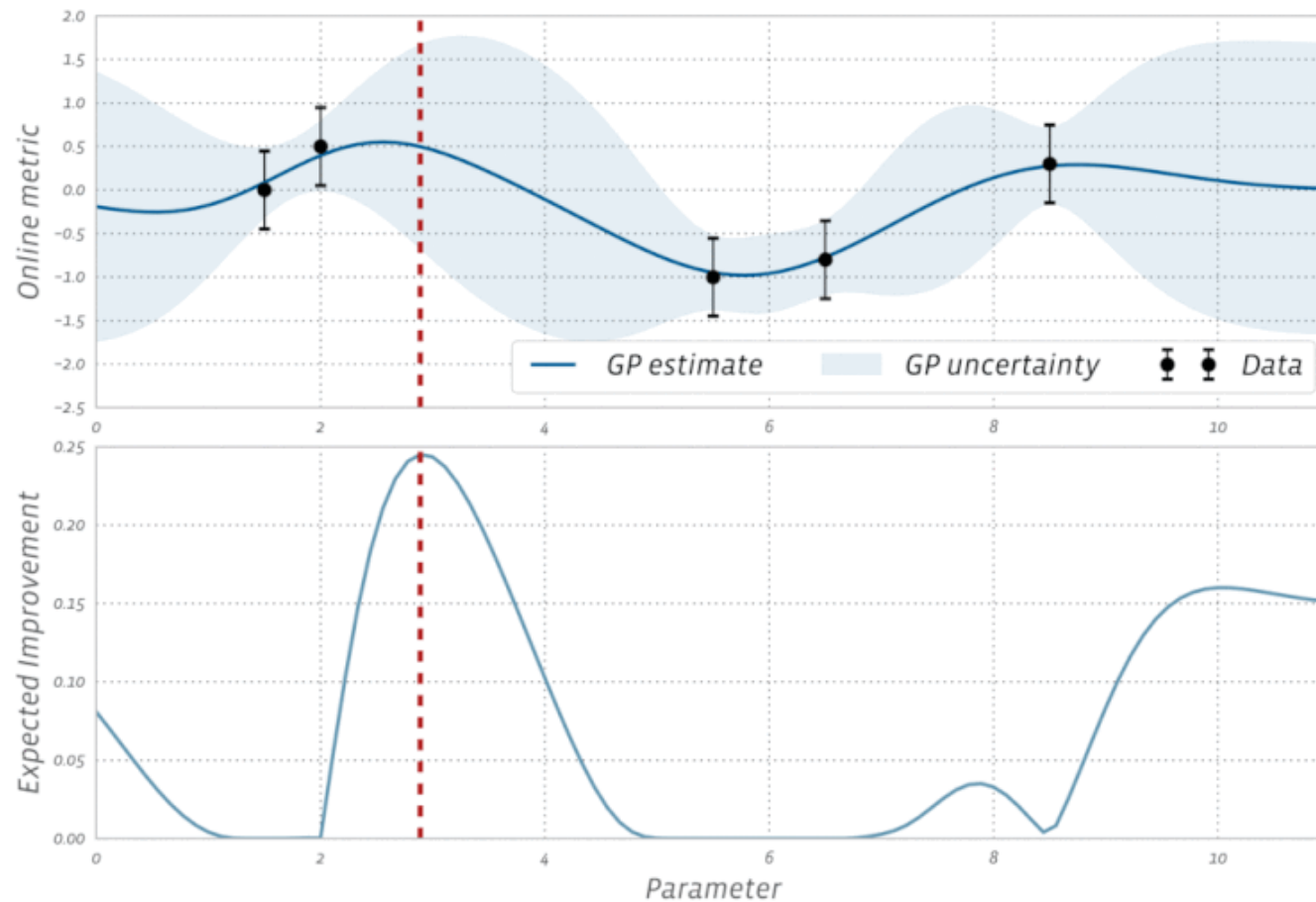
Bayesian Optimisation

- When the number of hyper-parameters is not small or some of the parameters are continuous, using large factorial designs (e.g., “grid search”) or global optimization techniques for optimization require more evaluations than is practically feasible.
- Bayesian optimization (BO) allows us to tune parameters in relatively few iterations by building a smooth model from an initial set of parameterizations (referred to as the "surrogate model") in order to predict the outcomes for as yet unexplored parameterizations.

Using Gaussian Processes as surrogate models



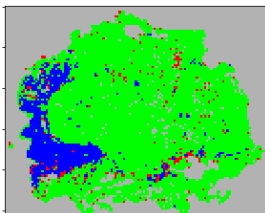
Maximise “Expected Improvement” (EI) to optimize parameters with minimum iterations



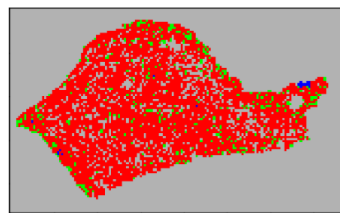
Do you need more data?

- Sometimes you will just need more data, rather than experimenting with the model
- Is your **training performance** acceptable?
 - If not, it is not using the training data available, so more data is unlikely to help. Try a larger model (higher capacity) or optimising hyperparameters. Maybe your data is very noisy?
- If yes, then is your performance on test data acceptable?
 - If it just cannot be improved any more ... maybe you just need more data.

Benign Lipoma

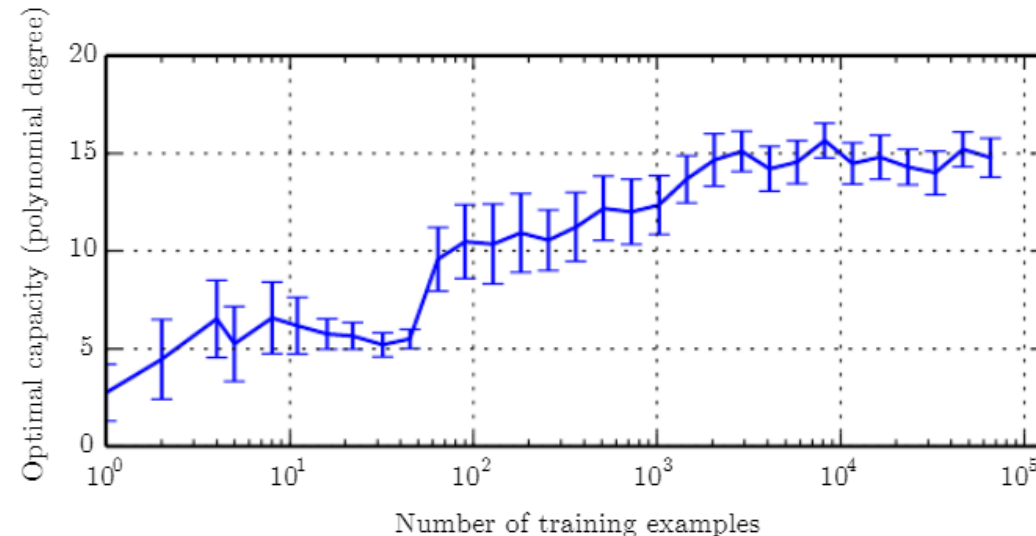
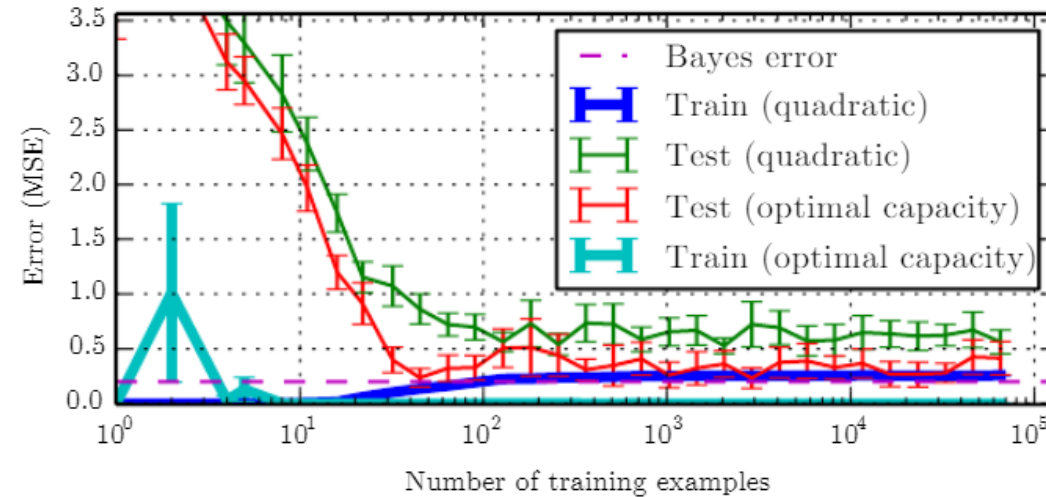


Malignant Liposarcoma



Can predict how much data using “learning curves”

Effect of training dataset size on train & test error



Background Reading

- Understanding Deep Learning (Simon Prince):
 - Chapter 5 describes how loss functions work in much more detail.
 - Chapter 8 discusses measuring performance and covers much of this material.
- Deep Learning (Goodfellow et al., 2018):
 - Chapter 6 provides a lot of the formal theory given in this lecture.
 - Chapter 11 covers a lot of the practical ML guidelines given in this lecture: